

Implementation Guide

Production-Ready Data Ingestion Implementation

Table of Contents

1. [Development Environment Setup](#)
 2. [Kafka Implementation](#)
 3. [Flink Stream Processing](#)
 4. [Spark Batch/Stream Processing](#)
 5. [CDC with Debezium](#)
 6. [Airflow Orchestration](#)
 7. [Monitoring & Observability](#)
 8. [Error Handling & Recovery](#)
 9. [Testing Strategies](#)
 10. [CI/CD Pipelines](#)
-

1. Development Environment Setup {#dev-environment}

Local Development Stack (Docker Compose)

File: `docker-compose.yml`

```
version: '3.8'

services:
  # Zookeeper for Kafka
  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    hostname: zookeeper
    container_name: zookeeper
    ports:
      - "2181:2181"
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    volumes:
      - zookeeper-data:/var/lib/zookeeper/data
      - zookeeper-logs:/var/lib/zookeeper/log

  # Kafka broker
  kafka:
    image: confluentinc/cp-kafka:7.5.0
    hostname: kafka
    container_name: kafka
    depends_on:
```

```

    - zookeeper
ports:
  - "9092:9092"
  - "9101:9101"
environment:
  KAFKA_BROKER_ID: 1
  KAFKA_ZOOKEEPER_CONNECT: 'zookeeper:2181'
  KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
  KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092
  KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
  KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
  KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
  KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
  KAFKA_JMX_PORT: 9101
  KAFKA_JMX_HOSTNAME: localhost
  KAFKA_AUTO_CREATE_TOPICS_ENABLE: 'true'
volumes:
  - kafka-data:/var/lib/kafka/data

# Schema Registry
schema-registry:
  image: confluentinc/cp-schema-registry:7.5.0
  hostname: schema-registry
  container_name: schema-registry
  depends_on:
    - kafka
  ports:
    - "8081:8081"
  environment:
    SCHEMA_REGISTRY_HOST_NAME: schema-registry
    SCHEMA_REGISTRY_KAFKASTORE_BOOTSTRAP_SERVERS: 'kafka:29092'
    SCHEMA_REGISTRY_LISTENERS: http://0.0.0.0:8081

# Kafka Connect
kafka-connect:
  image: confluentinc/cp-kafka-connect:7.5.0
  hostname: kafka-connect
  container_name: kafka-connect
  depends_on:
    - kafka
    - schema-registry
  ports:
    - "8083:8083"
  environment:
    CONNECT_BOOTSTRAP_SERVERS: 'kafka:29092'
    CONNECT_REST_ADVERTISED_HOST_NAME: kafka-connect
    CONNECT_GROUP_ID: compose-connect-group
    CONNECT_CONFIG_STORAGE_TOPIC: docker-connect-configs
    CONNECT_CONFIG_STORAGE_REPLICATION_FACTOR: 1
    CONNECT_OFFSET_STORAGE_TOPIC: docker-connect-offsets
    CONNECT_OFFSET_STORAGE_REPLICATION_FACTOR: 1
    CONNECT_STATUS_STORAGE_TOPIC: docker-connect-status

```

```

CONNECT_STATUS_STORAGE_REPLICATION_FACTOR: 1
CONNECT_KEY_CONVERTER: org.apache.kafka.connect.storage.StringConverter
CONNECT_VALUE_CONVERTER: io.confluent.connect.avro.AvroConverter
CONNECT_VALUE_CONVERTER_SCHEMA_REGISTRY_URL: http://schema-registry:8081
CONNECT_PLUGIN_PATH: "/usr/share/java,/usr/share/confluent-hub-components"
volumes:
  - ./connectors:/usr/share/confluent-hub-components

# PostgreSQL (source database)
postgres:
  image: postgres:15
  hostname: postgres
  container_name: postgres
  ports:
    - "5432:5432"
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: sourcedb
  command:
    - "postgres"
    - "-c"
    - "wal_level=logical"
    - "-c"
    - "max_replication_slots=4"
    - "-c"
    - "max_wal_senders=4"
  volumes:
    - postgres-data:/var/lib/postgresql/data
    - ./init-scripts:/docker-entrypoint-initdb.d

# Flink JobManager
flink-jobmanager:
  image: flink:1.18-scala_2.12
  hostname: flink-jobmanager
  container_name: flink-jobmanager
  ports:
    - "8088:8081"
  command: jobmanager
  environment:
    - |
      FLINK_PROPERTIES=
      jobmanager.rpc.address: flink-jobmanager
      parallelism.default: 2
      state.backend: rocksdb
      state.checkpoints.dir: file:///tmp/flink-checkpoints
      state.savepoints.dir: file:///tmp/flink-savepoints
  volumes:
    - flink-checkpoints:/tmp/flink-checkpoints
    - flink-savepoints:/tmp/flink-savepoints

# Flink TaskManager
flink-taskmanager:
  image: flink:1.18-scala_2.12

```

```
hostname: flink-taskmanager
container_name: flink-taskmanager
depends_on:
  - flink-jobmanager
command: taskmanager
scale: 2
environment:
  - |
    FLINK_PROPERTIES=
    jobmanager.rpc.address: flink-jobmanager
    taskmanager.numberOfTaskSlots: 4
    parallelism.default: 2

# Elasticsearch
elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:8.11.0
  container_name: elasticsearch
  environment:
    - discovery.type=single-node
    - xpack.security.enabled=false
    - "ES_JAVA_OPTS=-Xms512m -Xmx512m"
  ports:
    - "9200:9200"
  volumes:
    - es-data:/usr/share/elasticsearch/data

# Kibana
kibana:
  image: docker.elastic.co/kibana/kibana:8.11.0
  container_name: kibana
  ports:
    - "5601:5601"
  environment:
    ELASTICSEARCH_URL: http://elasticsearch:9200
    ELASTICSEARCH_HOSTS: '["http://elasticsearch:9200"]'
  depends_on:
    - elasticsearch

# Redis (for caching/state)
redis:
  image: redis:7-alpine
  container_name: redis
  ports:
    - "6379:6379"
  volumes:
    - redis-data:/data

# Prometheus
prometheus:
  image: prom/prometheus:latest
  container_name: prometheus
  ports:
    - "9090:9090"
  volumes:
```

```

    - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    - prometheus-data:/prometheus
  command:
    - '--config.file=/etc/prometheus/prometheus.yml'
    - '--storage.tsdb.path=/prometheus'

# Grafana
grafana:
  image: grafana/grafana:latest
  container_name: grafana
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=admin
    - GF_USERS_ALLOW_SIGN_UP=false
  volumes:
    - grafana-data:/var/lib/grafana
    - ./grafana/provisioning:/etc/grafana/provisioning
  depends_on:
    - prometheus

volumes:
  zookeeper-data:
  zookeeper-logs:
  kafka-data:
  postgres-data:
  flink-checkpoints:
  flink-savepoints:
  es-data:
  redis-data:
  prometheus-data:
  grafana-data:

```

Start the environment:

```

# Start all services
docker-compose up -d

# Check status
docker-compose ps

# View logs
docker-compose logs -f kafka

# Stop all services
docker-compose down

# Clean everything (including volumes)
docker-compose down -v

```

2. Kafka Implementation {#kafka-implementation}

Production Kafka Configuration

File: `kafka-config/server.properties`

```
# Broker Settings
broker.id=1
listeners=PLAINTEXT://:9092
advertised.listeners=PLAINTEXT://kafka-broker-1.example.com:9092
log.dirs=/var/lib/kafka/data

# Zookeeper
zookeeper.connect=zk1:2181,zk2:2181,zk3:2181
zookeeper.connection.timeout.ms=18000

# Replication
default.replication.factor=3
min.insync.replicas=2
unclean.leader.election.enable=false
auto.create.topics.enable=false

# Retention
log.retention.hours=168
log.retention.bytes=1073741824
log.segment.bytes=1073741824
log.cleanup.policy=delete

# Performance Tuning
num.network.threads=8
num.io.threads=16
socket.send.buffer.bytes=102400
socket.receive.buffer.bytes=102400
socket.request.max.bytes=104857600

# Compression
compression.type=lz4

# Transactions
transaction.state.log.replication.factor=3
transaction.state.log.min.isr=2
```

Java Producer Implementation

File: `KafkaProducerExample.java`

```
package com.example.kafka;

import org.apache.kafka.clients.producer.*;
import org.apache.kafka.common.serialization.StringSerializer;
```

```
import com.fasterxml.jackson.databind.ObjectMapper;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.util.Properties;
import java.util.concurrent.Future;

public class RobustKafkaProducer {

    private static final Logger logger =
        LoggerFactory.getLogger(RobustKafkaProducer.class);
    private final KafkaProducer<String, String> producer;
    private final ObjectMapper objectMapper;

    public RobustKafkaProducer(String bootstrapServers) {
        Properties props = new Properties();

        // Connection settings
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
        props.put(ProducerConfig.CLIENT_ID_CONFIG, "robust-producer-1");

        // Serialization
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
            StringSerializer.class.getName());
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
            StringSerializer.class.getName());

        // Idempotence (exactly-once semantics)
        props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, "true");
        props.put(ProducerConfig.ACKS_CONFIG, "all");
        props.put(ProducerConfig.RETRIES_CONFIG, Integer.MAX_VALUE);
        props.put(ProducerConfig.MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION, "5");

        // Compression
        props.put(ProducerConfig.COMPRESSION_TYPE_CONFIG, "lz4");

        // Batching for throughput
        props.put(ProducerConfig.LINGER_MS_CONFIG, "10");
        props.put(ProducerConfig.BATCH_SIZE_CONFIG, "32768");

        // Buffer memory
        props.put(ProducerConfig.BUFFER_MEMORY_CONFIG, "67108864"); // 64MB

        // Timeouts
        props.put(ProducerConfig.REQUEST_TIMEOUT_MS_CONFIG, "30000");
        props.put(ProducerConfig.DELIVERY_TIMEOUT_MS_CONFIG, "120000");

        this.producer = new KafkaProducer<>(props);
        this.objectMapper = new ObjectMapper();

        // Shutdown hook
        Runtime.getRuntime().addShutdownHook(new Thread(this::close));
    }
}
```

```

/**
 * Send message asynchronously with callback
 */
public Future<RecordMetadata> sendAsync(String topic, String key, Object
value) {
    try {
        String jsonValue = objectMapper.writeValueAsString(value);
        ProducerRecord<String, String> record = new ProducerRecord<>(topic,
key, jsonValue);

        return producer.send(record, (metadata, exception) -> {
            if (exception != null) {
                logger.error("Error sending message to topic {}: {}", topic,
exception.getMessage(), exception);
                // Handle error (e.g., send to DLQ, alert, retry)
                handleSendError(record, exception);
            } else {
                logger.debug("Message sent successfully: topic={}, partition=
{}", offset={}");
                metadata.topic(), metadata.partition(),
metadata.offset());
            }
        });
    } catch (Exception e) {
        logger.error("Error serializing message", e);
        throw new RuntimeException("Failed to send message", e);
    }
}

/**
 * Send message synchronously (blocks until acknowledged)
 */
public RecordMetadata sendSync(String topic, String key, Object value) {
    try {
        Future<RecordMetadata> future = sendAsync(topic, key, value);
        return future.get(); // Block until complete
    } catch (Exception e) {
        logger.error("Error sending message synchronously", e);
        throw new RuntimeException("Failed to send message", e);
    }
}

/**
 * Send message with custom headers
 */
public Future<RecordMetadata> sendWithHeaders(String topic, String key, Object
value,
                                           Map<String, String> headers) {
    try {
        String jsonValue = objectMapper.writeValueAsString(value);
        ProducerRecord<String, String> record = new ProducerRecord<>(topic,
key, jsonValue);

```

```

        // Add headers
        headers.forEach((k, v) ->
            record.headers().add(k, v.getBytes(StandardCharsets.UTF_8))
        );

        return producer.send(record);

    } catch (Exception e) {
        throw new RuntimeException("Failed to send message with headers", e);
    }
}

/**
 * Handle send errors
 */
private void handleSendError(ProducerRecord<String, String> record, Exception
exception) {
    // Strategy 1: Send to Dead Letter Queue
    try {
        String dlqTopic = record.topic() + ".dlq";
        ProducerRecord<String, String> dlqRecord = new ProducerRecord<>(
            dlqTopic,
            record.key(),
            record.value()
        );
        dlqRecord.headers().add("error", exception.getMessage().getBytes());
        dlqRecord.headers().add("original_topic", record.topic().getBytes());

        producer.send(dlqRecord);
        logger.info("Message sent to DLQ: {}", dlqTopic);

    } catch (Exception e) {
        logger.error("Failed to send to DLQ", e);
        // Strategy 2: Persist to local disk for manual recovery
        persistToLocalDisk(record);
    }
}

/**
 * Persist failed messages to local disk
 */
private void persistToLocalDisk(ProducerRecord<String, String> record) {
    try {
        Path failedDir = Paths.get("failed_messages");
        Files.createDirectories(failedDir);

        String filename = String.format("%s_%d.json",
            record.topic(), System.currentTimeMillis());
        Path filePath = failedDir.resolve(filename);

        Map<String, Object> failedMessage = Map.of(
            "topic", record.topic(),
            "key", record.key(),
            "value", record.value(),

```

```

        "timestamp", System.currentTimeMillis()
    );

    Files.writeString(filePath,
        objectMapper.writeValueAsString(failedMessage));

    logger.info("Failed message persisted to: {}", filePath);

} catch (Exception e) {
    logger.error("Failed to persist message to disk", e);
}

}

/**
 * Flush and close producer
 */
public void close() {
    logger.info("Closing Kafka producer...");
    producer.flush();
    producer.close();
    logger.info("Kafka producer closed");
}

/**
 * Example usage
 */
public static void main(String[] args) {
    RobustKafkaProducer producer = new RobustKafkaProducer("localhost:9092");

    // Send messages
    for (int i = 0; i < 100; i++) {
        Event event = new Event("user-" + i, "page_view",
            System.currentTimeMillis());
        producer.sendAsync("user-events", event.getUserId(), event);
    }

    producer.close();
}

// Event POJO
class Event {
    private String userId;
    private String eventType;
    private long timestamp;

    // Constructor, getters, setters...
}

```

Python Producer Implementation

File: `kafka_producer.py`

```

from confluent_kafka import Producer, KafkaError
from confluent_kafka.admin import AdminClient, NewTopic
import json
import logging
from typing import Dict, Any, Optional
from datetime import datetime
import time

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class RobustKafkaProducer:
    """
    Production-ready Kafka producer with error handling and monitoring
    """

    def __init__(self, bootstrap_servers: str, **kwargs):
        """
        Initialize Kafka producer

        Args:
            bootstrap_servers: Comma-separated list of Kafka brokers
            **kwargs: Additional producer configuration
        """
        config = {
            'bootstrap.servers': bootstrap_servers,
            'client.id': 'robust-producer-python',

            # Idempotence (exactly-once)
            'enable.idempotence': True,
            'acks': 'all',
            'retries': 2147483647,
            'max.in.flight.requests.per.connection': 5,

            # Compression
            'compression.type': 'lz4',

            # Batching
            'linger.ms': 10,
            'batch.size': 32768,

            # Timeouts
            'request.timeout.ms': 30000,
            'delivery.timeout.ms': 120000,

            # Callbacks
            'error_cb': self._error_callback,
            'stats_cb': self._stats_callback,
            'statistics.interval.ms': 60000,
        }

        # Merge with user config
        config.update(kwargs)

```

```

        self.producer = Producer(config)
        self.messages_sent = 0
        self.messages_failed = 0

        logger.info(f"Kafka producer initialized: {bootstrap_servers}")

    def _error_callback(self, err):
        """Global error callback"""
        logger.error(f"Kafka producer error: {err}")

    def _stats_callback(self, stats_json):
        """Statistics callback"""
        stats = json.loads(stats_json)
        logger.info(f"Producer stats: messages={stats.get('txmsgs', 0)}, "
                    f"bytes={stats.get('txmsg_bytes', 0)}")

    def send(self, topic: str, value: Dict[Any, Any],
             key: Optional[str] = None,
             headers: Optional[Dict[str, str]] = None,
             on_delivery: Optional[callable] = None) -> None:
        """
        Send message to Kafka

        Args:
            topic: Kafka topic name
            value: Message payload (will be JSON serialized)
            key: Message key (optional)
            headers: Message headers (optional)
            on_delivery: Callback function (optional)
        """
        try:
            # Serialize value
            serialized_value = json.dumps(value).encode('utf-8')

            # Serialize key
            serialized_key = key.encode('utf-8') if key else None

            # Prepare headers
            kafka_headers = []
            if headers:
                kafka_headers = [(k, v.encode('utf-8')) for k, v in
                                headers.items()]

            # Default delivery callback
            if on_delivery is None:
                on_delivery = self._default_delivery_callback

            # Send message
            self.producer.produce(
                topic=topic,
                value=serialized_value,
                key=serialized_key,
                headers=kafka_headers,

```

```

        on_delivery=on_delivery
    )

    # Poll for callbacks
    self.producer.poll(0)

except BufferError:
    logger.warning("Producer queue full, waiting...")
    # Wait for queue to drain
    self.producer.poll(1)
    # Retry
    self.send(topic, value, key, headers, on_delivery)

except Exception as e:
    logger.error(f"Error sending message: {e}")
    self.messages_failed += 1
    self._handle_send_error(topic, key, value, e)
    raise

def _default_delivery_callback(self, err, msg):
    """Default delivery report callback"""
    if err:
        logger.error(f"Message delivery failed: {err}")
        self.messages_failed += 1
        self._handle_delivery_error(msg, err)
    else:
        logger.debug(f"Message delivered: topic={msg.topic()}, "
                    f"partition={msg.partition()}, offset={msg.offset()}")
        self.messages_sent += 1

def _handle_send_error(self, topic: str, key: str, value: dict, error:
Exception):
    """Handle send errors"""
    # Write to DLQ
    dlq_topic = f"{topic}.dlq"
    dlq_message = {
        'original_topic': topic,
        'original_key': key,
        'original_value': value,
        'error': str(error),
        'timestamp': datetime.now().isoformat()
    }

    try:
        self.producer.produce(
            topic=dlq_topic,
            value=json.dumps(dlq_message).encode('utf-8')
        )
        logger.info(f"Message sent to DLQ: {dlq_topic}")
    except Exception as e:
        logger.error(f"Failed to send to DLQ: {e}")
        self._persist_to_disk(dlq_message)

def _handle_delivery_error(self, msg, error):

```

```

        """Handle delivery errors"""
        logger.error(f"Delivery error for topic {msg.topic()}: {error}")
        # Additional error handling logic here

def _persist_to_disk(self, message: dict):
    """Persist failed messages to local disk"""
    import os
    from pathlib import Path

    failed_dir = Path("failed_messages")
    failed_dir.mkdir(exist_ok=True)

    filename = f"{message['original_topic']}_{int(time.time() * 1000)}.json"
    filepath = failed_dir / filename

    with open(filepath, 'w') as f:
        json.dump(message, f, indent=2)

    logger.info(f"Failed message persisted to: {filepath}")

def flush(self, timeout: float = 30.0):
    """
    Flush pending messages

    Args:
        timeout: Maximum time to wait (seconds)
    """
    remaining = self.producer.flush(timeout)
    if remaining > 0:
        logger.warning(f"{remaining} messages remaining in queue after flush")
    else:
        logger.info("All messages flushed successfully")

def close(self):
    """Close producer"""
    logger.info("Closing Kafka producer...")
    self.flush()
    logger.info(f"Stats: sent={self.messages_sent}, failed={self.messages_failed}")

def __enter__(self):
    return self

def __exit__(self, exc_type, exc_val, exc_tb):
    self.close()

# Example usage
if __name__ == "__main__":
    with RobustKafkaProducer('localhost:9092') as producer:

        # Send messages
        for i in range(100):
            event = {
                'user_id': f'user-{i}',

```

```

        'event_type': 'page_view',
        'timestamp': datetime.now().isoformat(),
        'page': '/home'
    }

    producer.send(
        topic='user-events',
        key=event['user_id'],
        value=event,
        headers={'source': 'web_app'}
    )

    print("Messages sent")

```

Kafka Consumer Implementation

File: `KafkaConsumerExample.java`

```

package com.example.kafka;

import org.apache.kafka.clients.consumer.*;
import org.apache.kafka.common.TopicPartition;
import org.apache.kafka.common.serialization.StringDeserializer;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.time.Duration;
import java.util.*;
import java.util.concurrent.atomic.AtomicBoolean;

public class RobustKafkaConsumer {

    private static final Logger logger =
        LoggerFactory.getLogger(RobustKafkaConsumer.class);
    private final KafkaConsumer<String, String> consumer;
    private final AtomicBoolean running = new AtomicBoolean(true);

    public RobustKafkaConsumer(String bootstrapServers, String groupId) {
        Properties props = new Properties();

        // Connection
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
        props.put(ConsumerConfig.GROUP_ID_CONFIG, groupId);
        props.put(ConsumerConfig.CLIENT_ID_CONFIG, "robust-consumer-1");

        // Deserialization
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
            StringDeserializer.class.getName());
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
            StringDeserializer.class.getName());
    }

```

```

// Offset management
props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, "false");
props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");

// Fetch settings
props.put(ConsumerConfig.FETCH_MIN_BYTES_CONFIG, "1");
props.put(ConsumerConfig.FETCH_MAX_WAIT_MS_CONFIG, "500");
props.put(ConsumerConfig.MAX_POLL_RECORDS_CONFIG, "500");

// Session management
props.put(ConsumerConfig.SESSION_TIMEOUT_MS_CONFIG, "30000");
props.put(ConsumerConfig.HEARTBEAT_INTERVAL_MS_CONFIG, "10000");
props.put(ConsumerConfig.MAX_POLL_INTERVAL_MS_CONFIG, "300000");

this.consumer = new KafkaConsumer<>(props);

// Shutdown hook
Runtime.getRuntime().addShutdownHook(new Thread(this::shutdown));
}

/**
 * Start consuming messages
 */
public void consume(String topic, MessageProcessor processor) {
    try {
        consumer.subscribe(Collections.singletonList(topic));
        logger.info("Started consuming from topic: {}", topic);

        while (running.get()) {
            ConsumerRecords<String, String> records =
consumer.poll(Duration.ofMillis(100));

            if (!records.isEmpty()) {
                logger.debug("Polled {} records", records.count());
                processRecords(records, processor);

                // Manual commit after processing
                try {
                    consumer.commitSync();
                } catch (CommitFailedException e) {
                    logger.error("Failed to commit offsets", e);
                }
            }
        }

    } catch (Exception e) {
        logger.error("Error in consumer loop", e);
    } finally {
        close();
    }
}

/**
 * Process records with error handling

```

```

    */
    private void processRecords(ConsumerRecords<String, String> records,
                               MessageProcessor processor) {
        Map<TopicPartition, OffsetAndMetadata> offsetsToCommit = new HashMap<>();

        for (ConsumerRecord<String, String> record : records) {
            try {
                // Process message
                processor.process(record);

                // Track offset
                TopicPartition partition = new TopicPartition(record.topic(),
record.partition());
                offsetsToCommit.put(partition,
                    new OffsetAndMetadata(record.offset() + 1));

            } catch (Exception e) {
                logger.error("Error processing record: topic={}, partition={},
offset={},",
                    record.topic(), record.partition(), record.offset(), e);

                // Handle error
                handleProcessingError(record, e);

                // Decide: Skip or Stop?
                // Option 1: Skip and continue
                continue;

                // Option 2: Stop processing (uncomment to use)
                // throw new RuntimeException("Failed to process record", e);
            }
        }

        // Commit processed offsets
        if (!offsetsToCommit.isEmpty()) {
            try {
                consumer.commitSync(offsetsToCommit);
            } catch (CommitFailedException e) {
                logger.error("Failed to commit offsets", e);
            }
        }
    }

    /**
     * Handle processing errors
     */
    private void handleProcessingError(ConsumerRecord<String, String> record,
Exception error) {
        // Send to DLQ
        try {
            // In production, use a separate producer for DLQ
            logger.info("Sending failed message to DLQ");
            // sendToDLQ(record, error);
        } catch (Exception e) {

```

```

        logger.error("Failed to send to DLQ", e);
    }
}

/**
 * Graceful shutdown
 */
public void shutdown() {
    logger.info("Shutting down consumer...");
    running.set(false);
}

/**
 * Close consumer
 */
private void close() {
    logger.info("Closing consumer...");
    consumer.close();
}

/**
 * Example usage
 */
public static void main(String[] args) {
    RobustKafkaConsumer consumer = new RobustKafkaConsumer(
        "localhost:9092",
        "my-consumer-group"
    );

    consumer.consume("user-events", record -> {
        logger.info("Processing: key={}, value={}", record.key(),
record.value());
        // Process message here
    });
}

// Functional interface for message processing
@FunctionalInterface
interface MessageProcessor {
    void process(ConsumerRecord<String, String> record) throws Exception;
}

```

3. Flink Stream Processing {#flink-implementation}

Flink Job Structure

File: `FlinkStreamingJob.java`

```
package com.example.flink;

import org.apache.flink.api.common.eventtime.WatermarkStrategy;
import org.apache.flink.api.common.functions.MapFunction;
import org.apache.flink.api.common.serialization.SimpleStringSchema;
import org.apache.flink.connector.kafka.source.KafkaSource;
import org.apache.flink.connector.kafka.source.enumerator.initializer.OffsetsInitializer;
import org.apache.flink.connector.kafka.sink.KafkaSink;
import org.apache.flink.connector.kafka.sink.KafkaRecordSerializationSchema;
import org.apache.flink.streaming.api.datastream.DataStream;
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import org.apache.flink.streaming.api.windowing.assigners.TumblingEventTimeWindows;
import org.apache.flink.streaming.api.windowing.time.Time;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.node.ObjectNode;

import java.time.Duration;

public class UserEventProcessor {

    public static void main(String[] args) throws Exception {

        // Set up execution environment
        final StreamExecutionEnvironment env =
            StreamExecutionEnvironment.getExecutionEnvironment();

        // Enable checkpointing (every 60 seconds)
        env.enableCheckpointing(60000);
        env.getCheckpointConfig().setMinPauseBetweenCheckpoints(30000);
        env.getCheckpointConfig().setCheckpointTimeout(180000);
        env.getCheckpointConfig().setMaxConcurrentCheckpoints(1);

        // Kafka source
        KafkaSource<String> source = KafkaSource.<String>builder()
            .setBootstrapServers("localhost:9092")
            .setTopics("user-events")
            .setGroupId("flink-consumer-group")
            .setStartingOffsets(OffsetsInitializer.earliest())
            .setValueOnlyDeserializer(new SimpleStringSchema())
            .build();

        // Read from Kafka
        DataStream<String> events = env
            .fromSource(source, WatermarkStrategy.noWatermarks(), "Kafka Source");

        // Parse JSON
        DataStream<Event> parsedEvents = events
            .map(new JsonParser())
            .name("Parse JSON");

        // Assign timestamps and watermarks
```

```

        DataStream<Event> eventsWithTimestamps = parsedEvents
            .assignTimestampsAndWatermarks(
                WatermarkStrategy
                    .<Event>forBoundedOutOfOrderness(Duration.ofSeconds(10))
                    .withTimestampAssigner((event, timestamp) ->
event.getTimestamp())
            );

// Windowed aggregation (5-minute tumbling windows)
DataStream<WindowedMetrics> metrics = eventsWithTimestamps
    .keyBy(Event::getUserId)
    .window(TumblingEventTimeWindows.of(Time.minutes(5)))
    .aggregate(new EventCountAggregator())
    .name("Aggregate Events");

// Convert to JSON for output
DataStream<String> output = metrics
    .map(metric -> new ObjectMapper().writeValueAsString(metric))
    .name("Serialize to JSON");

// Kafka sink
KafkaSink<String> sink = KafkaSink.<String>builder()
    .setBootstrapServers("localhost:9092")
    .setRecordSerializer(KafkaRecordSerializationSchema.builder()
        .setTopic("user-metrics")
        .setValueSerializationSchema(new SimpleStringSchema())
        .build()
    )
    .build();

// Write to Kafka
output.sinkTo(sink).name("Kafka Sink");

// Execute job
env.execute("User Event Processor");
}

// JSON parser
public static class JsonParser implements MapFunction<String, Event> {
    private final ObjectMapper mapper = new ObjectMapper();

    @Override
    public Event map(String value) throws Exception {
        ObjectNode node = mapper.readValue(value, ObjectNode.class);
        return new Event(
            node.get("user_id").asText(),
            node.get("event_type").asText(),
            node.get("timestamp").asLong()
        );
    }
}

}

// Event POJO

```

```
class Event {
    private String userId;
    private String eventType;
    private long timestamp;

    // Constructor, getters, setters, toString...
}

// Windowed metrics POJO
class WindowedMetrics {
    private String userId;
    private long windowStart;
    private long windowEnd;
    private long eventCount;

    // Constructor, getters, setters, toString...
}
```

Deployment:

```
# Build JAR
mvn clean package

# Submit to Flink cluster
flink run -c com.example.flink.UserEventProcessor \
    target/flink-streaming-job-1.0.jar

# Check job status
flink list

# Cancel job
flink cancel <job-id>

# Create savepoint
flink savepoint <job-id> s3://savepoints/

# Resume from savepoint
flink run -s s3://savepoints/savepoint-123 \
    -c com.example.flink.UserEventProcessor \
    target/flink-streaming-job-1.0.jar
```

4. Spark Batch/Stream Processing {#spark-implementation}

Spark Structured Streaming

File: `spark_streaming_job.py`

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
from pyspark.sql.streaming import StreamingQuery
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class SparkStreamProcessor:
    """
    Production Spark Structured Streaming job
    """

    def __init__(self, app_name: str):
        """Initialize Spark session"""
        self.spark = SparkSession.builder \
            .appName(app_name) \
            .config("spark.sql.shuffle.partitions", "200") \
            .config("spark.sql.streaming.checkpointLocation", "s3://checkpoints/") \
            .config("spark.sql.streaming.schemaInference", "true") \
            .config("spark.streaming.stopGracefullyOnShutdown", "true") \
            .config("spark.sql.adaptive.enabled", "true") \
            .getOrCreate()

        self.spark.sparkContext.setLogLevel("WARN")
        logger.info(f"Spark session initialized: {app_name}")

    def create_kafka_stream(self, topic: str, bootstrap_servers: str):
        """
        Create Kafka streaming DataFrame

        Args:
            topic: Kafka topic name
            bootstrap_servers: Kafka brokers

        Returns:
            Streaming DataFrame
        """
        return self.spark.readStream \
            .format("kafka") \
            .option("kafka.bootstrap.servers", bootstrap_servers) \
            .option("subscribe", topic) \
            .option("startingOffsets", "latest") \
            .option("failOnDataLoss", "false") \
            .option("maxOffsetsPerTrigger", "10000") \
            .load()

    def parse_events(self, kafka_df):
        """Parse JSON events from Kafka"""
        schema = StructType([
            StructField("user_id", StringType()),

```

```

        StructField("event_type", StringType()),
        StructField("timestamp", TimestampType()),
        StructField("properties", MapType(StringType(), StringType()))
    ])

    return kafka_df \
        .selectExpr("CAST(value AS STRING) as json") \
        .select(from_json(col("json"), schema).alias("data")) \
        .select("data.*")

def process_stream(self):
    """Main processing logic"""

    # Read from Kafka
    raw_events = self.create_kafka_stream(
        topic="user-events",
        bootstrap_servers="localhost:9092"
    )

    # Parse JSON
    events = self.parse_events(raw_events)

    # Add processing timestamp
    events_with_metadata = events \
        .withColumn("processing_time", current_timestamp()) \
        .withColumn("date", to_date(col("timestamp")))

    # Windowed aggregation
    aggregated = events_with_metadata \
        .withWatermark("timestamp", "10 minutes") \
        .groupBy(
            window("timestamp", "5 minutes"),
            "user_id",
            "event_type"
        ) \
        .agg(
            count("*").alias("event_count"),
            collect_list("properties").alias("all_properties")
        )

    # Write to multiple sinks

    # Sink 1: Write to data lake (Parquet)
    data_lake_query = events_with_metadata \
        .writeStream \
        .outputMode("append") \
        .format("parquet") \
        .option("path", "s3://datalake/events/") \
        .option("checkpointLocation", "s3://checkpoints/datalake/") \
        .partitionBy("date", "event_type") \
        .trigger(processingTime="2 minutes") \
        .start()

    # Sink 2: Write aggregated metrics to Kafka

```

```

metrics_query = aggregated \
    .selectExpr("user_id as key", "to_json(struct(*)) as value") \
    .writeStream \
    .outputMode("update") \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("topic", "user-metrics") \
    .option("checkpointLocation", "s3://checkpoints/metrics/") \
    .trigger(processingTime="1 minute") \
    .start()

# Sink 3: Write to console (for debugging)
console_query = aggregated \
    .writeStream \
    .outputMode("update") \
    .format("console") \
    .option("truncate", "false") \
    .trigger(processingTime="30 seconds") \
    .start()

# Wait for all queries
self.spark.streams.awaitAnyTermination()

def stop(self):
    """Stop Spark session"""
    logger.info("Stopping Spark session...")
    self.spark.stop()

if __name__ == "__main__":
    processor = SparkStreamProcessor("User Event Processor")

    try:
        processor.process_stream()
    except KeyboardInterrupt:
        logger.info("Received interrupt signal")
    finally:
        processor.stop()

```

Submit to cluster:

```

# Local mode
spark-submit \
    --master local[4] \
    --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.0 \
    spark_streaming_job.py

# Cluster mode (YARN)
spark-submit \
    --master yarn \
    --deploy-mode cluster \
    --num-executors 10 \
    --executor-memory 4G \

```

```
--executor-cores 2 \
--driver-memory 2G \
--packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.0 \
--conf spark.sql.shuffle.partitions=200 \
--conf spark.streaming.stopGracefullyOnShutdown=true \
spark_streaming_job.py

# Kubernetes
spark-submit \
--master k8s://https://kubernetes.default.svc:443 \
--deploy-mode cluster \
--name user-event-processor \
--conf spark.executor.instances=5 \
--conf spark.kubernetes.container.image=my-spark:3.5.0 \
--conf spark.kubernetes.namespace=data-platform \
local:///opt/spark/work-dir/spark_streaming_job.py
```

[Document continues with sections 5-10 covering Debezium, Airflow, Monitoring, Error Handling, Testing, and CI/CD...]

Document Status

Current Size: ~45KB **Completion:** Part 1 of 2 **Sections Remaining:** 5-10

5. CDC with Debezium {#debezium-implementation}

Debezium PostgreSQL Connector

File: `debezium-postgres-connector.json`

```
{
  "name": "postgres-cdc-connector",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "postgres",
    "database.port": "5432",
    "database.user": "postgres",
    "database.password": "postgres",
    "database.dbname": "sourcedb",
    "database.server.name": "postgres_server",
    "table.include.list": "public.users,public.orders,public.products",
    "plugin.name": "pgoutput",

    "publication.autocreate.mode": "filtered",
    "slot.name": "debezium_slot",

    "key.converter": "org.apache.kafka.connect.json.JsonConverter",
    "key.converter.schemas.enable": "false",
    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
```

```

"value.converter.schemas.enable": "true",

"transforms": "unwrap",
"transforms.unwrap.type": "io.debezium.transforms.ExtractNewRecordState",
"transforms.unwrap.drop.tombstones": "false",
"transforms.unwrap.delete.handling.mode": "rewrite",
"transforms.unwrap.add.fields": "op,db,table,ts_ms",

"snapshot.mode": "initial",
"snapshot.locking.mode": "minimal",

"heartbeat.interval.ms": "10000",
"heartbeat.topics.prefix": "__debezium-heartbeat",

"errors.tolerance": "all",
"errors.log.enable": "true",
"errors.log.include.messages": "true",

"max.batch.size": "2048",
"max.queue.size": "8192",
"poll.interval.ms": "1000"
}
}

```

Deploy connector:

```

# Deploy to Kafka Connect
curl -X POST http://localhost:8083/connectors \
  -H "Content-Type: application/json" \
  -d @debezium-postgres-connector.json

# Check status
curl http://localhost:8083/connectors/postgres-cdc-connector/status | jq

# Pause connector
curl -X PUT http://localhost:8083/connectors/postgres-cdc-connector/pause

# Resume connector
curl -X PUT http://localhost:8083/connectors/postgres-cdc-connector/resume

# Delete connector
curl -X DELETE http://localhost:8083/connectors/postgres-cdc-connector

```

Processing Debezium CDC Events

File: `cdc_processor.py`

```

from confluent_kafka import Consumer
import json

```

```

from typing import Dict, Any
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class DebeziumCDCProcessor:
    """
    Process Debezium CDC events
    """

    def __init__(self, bootstrap_servers: str, group_id: str):
        self.consumer = Consumer({
            'bootstrap.servers': bootstrap_servers,
            'group.id': group_id,
            'auto.offset.reset': 'earliest',
            'enable.auto.commit': False
        })

    def process_cdc_event(self, message: Dict[str, Any]):
        """
        Process a single CDC event

        Debezium event structure:
        {
            "before": {...}, # State before change (null for INSERT)
            "after": {...}, # State after change (null for DELETE)
            "source": {...}, # Metadata
            "op": "c/u/d/r", # Operation: create/update/delete/read
            "ts_ms": 1234567890
        }
        """
        operation = message.get('op')
        before = message.get('before')
        after = message.get('after')
        source = message.get('source', {})

        table = source.get('table')
        timestamp = message.get('ts_ms')

        logger.info(f"CDC Event: table={table}, op={operation}, ts={timestamp}")

        if operation == 'c': # INSERT
            self.handle_insert(table, after)
        elif operation == 'u': # UPDATE
            self.handle_update(table, before, after)
        elif operation == 'd': # DELETE
            self.handle_delete(table, before)
        elif operation == 'r': # READ (snapshot)
            self.handle_snapshot(table, after)
        else:
            logger.warning(f"Unknown operation: {operation}")

    def handle_insert(self, table: str, record: Dict):

```

```

        """Handle INSERT event"""
        logger.info(f"INSERT into {table}: {record}")
        # Implement your logic: write to data lake, update cache, etc.

def handle_update(self, table: str, before: Dict, after: Dict):
    """Handle UPDATE event"""
    logger.info(f"UPDATE in {table}: {before} -> {after}")
    # Implement your logic

def handle_delete(self, table: str, record: Dict):
    """Handle DELETE event"""
    logger.info(f"DELETE from {table}: {record}")
    # Implement your logic

def handle_snapshot(self, table: str, record: Dict):
    """Handle snapshot READ event"""
    logger.info(f"SNAPSHOT from {table}: {record}")
    # Implement your logic

def start(self, topics: list):
    """Start consuming CDC events"""
    self.consumer.subscribe(topics)

    try:
        while True:
            msg = self.consumer.poll(1.0)

            if msg is None:
                continue
            if msg.error():
                logger.error(f"Consumer error: {msg.error()}")
                continue

            # Parse message
            try:
                event = json.loads(msg.value().decode('utf-8'))
                self.process_cdc_event(event)

                # Commit offset after processing
                self.consumer.commit(msg)

            except Exception as e:
                logger.error(f"Error processing message: {e}")
                # Handle error: send to DLQ, alert, etc.

        except KeyboardInterrupt:
            logger.info("Shutting down...")
        finally:
            self.consumer.close()

if __name__ == "__main__":
    processor = DebeziumCDCProcessor(
        bootstrap_servers='localhost:9092',
        group_id='cdc-processor-group'

```

```

)

# Subscribe to CDC topics
topics = [
    'postgres_server.public.users',
    'postgres_server.public.orders',
    'postgres_server.public.products'
]

processor.start(topics)

```

6. Airflow Orchestration {#airflow-implementation}

Production DAG Structure

File: dags/production_etl_pipeline.py

```

from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.amazon.aws.operators.s3 import S3CopyObjectOperator
from airflow.providers.amazon.aws.sensors.s3 import S3KeySensor
from airflow.providers.postgres.operators.postgres import PostgresOperator
from airflow.providers.apache.spark.operators.spark_submit import
SparkSubmitOperator
from airflow.utils.task_group import TaskGroup
from airflow.models import Variable
from airflow.utils.email import send_email
from datetime import datetime, timedelta
import logging

logger = logging.getLogger(__name__)

# Default arguments
default_args = {
    'owner': 'data-engineering',
    'depends_on_past': False,
    'email': ['data-eng@company.com'],
    'email_on_failure': True,
    'email_on_retry': False,
    'retries': 3,
    'retry_delay': timedelta(minutes=5),
    'retry_exponential_backoff': True,
    'max_retry_delay': timedelta(hours=1),
    'execution_timeout': timedelta(hours=2),
}

# DAG definition
with DAG(
    'production_etl_pipeline',
    default_args=default_args,

```

```

description='Production ETL pipeline with error handling',
schedule_interval='0 2 * * *', # 2 AM daily
start_date=datetime(2024, 1, 1),
catchup=False,
max_active_runs=1,
tags=['production', 'etl', 'daily'],
) as dag:

def send_start_notification(**context):
    """Send notification when DAG starts"""
    logger.info("DAG started")
    # Send Slack/email notification

def validate_input_data(**context):
    """Validate input data quality"""
    # Check row counts, schema, data quality rules
    logger.info("Validating input data...")
    # If validation fails, raise exception
    return True

def extract_from_postgres(**context):
    """Extract data from PostgreSQL"""
    from sqlalchemy import create_engine
    import pandas as pd

    engine = create_engine(Variable.get("postgres_conn_string"))

    # Extract data
    query = """
        SELECT *
        FROM orders
        WHERE created_at >= CURRENT_DATE - INTERVAL '1 day'
    """

    df = pd.read_sql(query, engine)

    # Save to S3
    s3_path = f"s3://staging/orders/{context['ds']}/orders.parquet"
    df.to_parquet(s3_path, index=False)

    logger.info(f"Extracted {len(df)} rows to {s3_path}")
    return s3_path

def transform_data(**context):
    """Transform data using PySpark"""
    # This would typically be a Spark job
    logger.info("Transforming data...")
    return True

def load_to_warehouse(**context):
    """Load to data warehouse"""
    logger.info("Loading to warehouse...")
    return True

```

```

def data_quality_checks(**context):
    """Run data quality checks"""
    logger.info("Running data quality checks...")
    # Check null rates, duplicates, etc.
    return True

def send_success_notification(**context):
    """Send success notification"""
    logger.info("DAG completed successfully")
    execution_date = context['execution_date']
    dag_run_duration = context['dag_run'].end_date -
context['dag_run'].start_date

    message = f"""
DAG: {context['dag'].dag_id}
Execution          Duration: {dag_run_duration}
Status: SUCCESS
    """

    send_email(
        to=['data-eng@company.com'],
        subject=f"[Airflow] {context['dag'].dag_id} - SUCCESS",
        html_content=message
    )

def send_failure_notification(**context):
    """Send failure notification"""
    logger.error("DAG failed")
    # Send Slack/email/PagerDuty alert

# Task 1: Start notification
start = PythonOperator(
    task_id='send_start_notification',
    python_callable=send_start_notification
)

# Task 2: Validate input
validate = PythonOperator(
    task_id='validate_input_data',
    python_callable=validate_input_data
)

# Task Group: Extract from multiple sources
with TaskGroup('extract_sources') as extract_group:

    extract_postgres = PythonOperator(
        task_id='extract_postgres',
        python_callable=extract_from_postgres
    )

    extract_api = PythonOperator(
        task_id='extract_api',
        python_callable=lambda: logger.info("Extract from API")
    )

```

```

extract_files = S3KeySensor(
    task_id='wait_for_files',
    bucket_name='raw-data',
    bucket_key='daily-files/{{ ds }}/*.csv',
    timeout=3600,
    poke_interval=60
)

# Task 4: Transform with Spark
transform = SparkSubmitOperator(
    task_id='transform_data',
    application='/opt/airflow/spark_jobs/transform.py',
    conn_id='spark_default',
    total_executor_cores=4,
    executor_cores=2,
    executor_memory='4g',
    driver_memory='2g',
    application_args=[
        '--input', 's3://staging/orders/{{ ds }}/',
        '--output', 's3://processed/orders/{{ ds }}/'
    ]
)

# Task 5: Load to warehouse
load = PostgresOperator(
    task_id='load_to_warehouse',
    postgres_conn_id='warehouse_postgres',
    sql="""
        COPY orders_staging
        FROM 's3://processed/orders/{{ ds }}/'
        IAM_ROLE 'arn:aws:iam::123456789:role/RedshiftS3ReadRole'
        FORMAT AS PARQUET;

        INSERT INTO orders
        SELECT * FROM orders_staging
        ON CONFLICT (order_id) DO UPDATE
        SET updated_at = EXCLUDED.updated_at;

        TRUNCATE orders_staging;
    """
)

# Task 6: Data quality checks
quality_checks = PythonOperator(
    task_id='data_quality_checks',
    python_callable=data_quality_checks
)

# Task 7: Success notification
success = PythonOperator(
    task_id='send_success_notification',
    python_callable=send_success_notification,
    trigger_rule='all_success'
)

```

```

    )

    # Task 8: Failure notification
    failure = PythonOperator(
        task_id='send_failure_notification',
        python_callable=send_failure_notification,
        trigger_rule='one_failed'
    )

    # Define task dependencies
    start >> validate >> extract_group >> transform >> load >> quality_checks
    quality_checks >> [success, failure]

```

Airflow Configuration (`airflow.cfg`):

```

[core]
dags_folder = /opt/airflow/dags
plugins_folder = /opt/airflow/plugins
load_examples = False
parallelism = 32
max_active_runs_per_dag = 16
dag_concurrency = 16
max_active_tasks_per_dag = 16

[scheduler]
scheduler_heartbeat_sec = 5
min_file_process_interval = 30
dag_dir_list_interval = 300
catchup_by_default = False
max_threads = 2

[webserver]
web_server_port = 8080
web_server_worker_timeout = 120
workers = 4
worker_refresh_interval = 30

[logging]
base_log_folder = /opt/airflow/logs
remote_logging = True
remote_log_conn_id = aws_default
remote_base_log_folder = s3://airflow-logs/

```

7. Monitoring & Observability {#monitoring}

Prometheus Metrics Collection

File: `prometheus/prometheus.yml`

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    cluster: 'production'
    env: 'prod'

# Alerting configuration
alerting:
  alertmanagers:
    - static_configs:
        - targets:
            - alertmanager:9093

# Load rules
rule_files:
  - 'alerts/*.yaml'

# Scrape configurations
scrape_configs:

  # Kafka exporters
  - job_name: 'kafka'
    static_configs:
      - targets:
          - 'kafka-1:9308'
          - 'kafka-2:9308'
          - 'kafka-3:9308'
    relabel_configs:
      - source_labels: [__address__]
        target_label: instance

  # Flink metrics
  - job_name: 'flink'
    static_configs:
      - targets:
          - 'flink-jobmanager:9249'
          - 'flink-taskmanager-1:9249'
          - 'flink-taskmanager-2:9249'

  # Application metrics
  - job_name: 'application'
    static_configs:
      - targets:
          - 'app-1:8000'
          - 'app-2:8000'
    metrics_path: '/metrics'

  # JMX exporter (for Kafka/Spark)
  - job_name: 'jmx'
    static_configs:
      - targets:
```

```
- 'kafka-1:7071'
- 'spark-master:8090'
```

Prometheus Alert Rules

File: `prometheus/alerts/data_ingestion.yml`

```
groups:
- name: data_ingestion_alerts
  interval: 30s
  rules:

    # Kafka lag alert
    - alert: KafkaConsumerLagHigh
      expr: kafka_consumer_group_lag > 10000
      for: 5m
      labels:
        severity: warning
        component: kafka
      annotations:
        summary: "High consumer lag detected"
        description: "Consumer group {{ $labels.group }} has lag of {{ $value }}
on topic {{ $labels.topic }}"

    # Kafka broker down
    - alert: KafkaBrokerDown
      expr: up{job="kafka"} == 0
      for: 2m
      labels:
        severity: critical
        component: kafka
      annotations:
        summary: "Kafka broker is down"
        description: "Kafka broker {{ $labels.instance }} is down"

    # Flink job failing
    - alert: FlinkJobFailed
      expr: flink_jobmanager_job_uptime == 0
      for: 1m
      labels:
        severity: critical
        component: flink
      annotations:
        summary: "Flink job has failed"
        description: "Flink job {{ $labels.job_name }} is not running"

    # Flink checkpoint failures
    - alert: FlinkCheckpointFailureRateHigh
      expr: rate(flink_jobmanager_job_numberOfFailedCheckpoints[5m]) > 0.1
      for: 5m
      labels:
```

```

        severity: warning
        component: flink
    annotations:
        summary: "High checkpoint failure rate"
        description: "Job {{ $labels.job_name }} has high checkpoint failure
rate"

# Data freshness alert
- alert: DataFreshnessLow
  expr: (time() - last_ingested_timestamp_seconds) > 3600
  for: 5m
  labels:
    severity: warning
    component: data_pipeline
  annotations:
    summary: "Data is stale"
    description: "No new data ingested in the last hour for {{
$labels.pipeline }}"

# Throughput drop
- alert: ThroughputDropped
  expr: rate(events_processed_total[5m]) < 100
  for: 10m
  labels:
    severity: warning
    component: data_pipeline
  annotations:
    summary: "Processing throughput dropped"
    description: "Throughput for {{ $labels.pipeline }} is below 100
events/sec"

```

Custom Application Metrics

File: `metrics.py`

```

from prometheus_client import Counter, Histogram, Gauge, Summary,
start_http_server
import time
import functools

# Define metrics
messages_processed_total = Counter(
    'messages_processed_total',
    'Total number of messages processed',
    ['pipeline', 'status']
)

processing_duration_seconds = Histogram(
    'processing_duration_seconds',
    'Time spent processing messages',
    ['pipeline'],

```

```

    buckets=[0.01, 0.05, 0.1, 0.5, 1, 2, 5, 10, 30, 60]
)

active_pipelines = Gauge(
    'active_pipelines',
    'Number of currently active pipelines',
    ['pipeline']
)

last_processed_timestamp = Gauge(
    'last_processed_timestamp_seconds',
    'Timestamp of last processed message',
    ['pipeline']
)

message_size_bytes = Summary(
    'message_size_bytes',
    'Size of processed messages',
    ['pipeline']
)

def track_processing_time(pipeline_name: str):
    """Decorator to track processing time"""
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            with
processing_duration_seconds.labels(pipeline=pipeline_name).time():
                return func(*args, **kwargs)
        return wrapper
    return decorator

class PipelineMetrics:
    """
    Metrics collector for data pipelines
    """

    def __init__(self, pipeline_name: str, port: int = 8000):
        self.pipeline_name = pipeline_name
        # Start metrics server
        start_http_server(port)

    def record_message_processed(self, status: str = 'success'):
        """Record a processed message"""
        messages_processed_total.labels(
            pipeline=self.pipeline_name,
            status=status
        ).inc()

    def record_message_size(self, size_bytes: int):
        """Record message size"""
        message_size_bytes.labels(
            pipeline=self.pipeline_name
        ).observe(size_bytes)

```

```

def update_last_processed(self):
    """Update last processed timestamp"""
    last_processed_timestamp.labels(
        pipeline=self.pipeline_name
    ).set(time.time())

def set_pipeline_active(self, active: bool):
    """Set pipeline active status"""
    active_pipelines.labels(
        pipeline=self.pipeline_name
    ).set(1 if active else 0)

@track_processing_time('example_pipeline')
def process_message(self, message: dict):
    """Example message processing with metrics"""
    try:
        # Process message
        result = self._do_processing(message)

        # Record metrics
        self.record_message_processed(status='success')
        self.record_message_size(len(str(message)))
        self.update_last_processed()

        return result

    except Exception as e:
        self.record_message_processed(status='failure')
        raise

# Example usage
if __name__ == "__main__":
    metrics = PipelineMetrics('example_pipeline', port=8000)
    metrics.set_pipeline_active(True)

    # Simulate processing
    while True:
        metrics.process_message({'data': 'example'})
        time.sleep(1)

```

Grafana Dashboard

File: grafana/dashboards/data-ingestion-dashboard.json

```

{
  "dashboard": {
    "title": "Data Ingestion Pipeline",
    "panels": [
      {
        "id": 1,

```

```

    "title": "Messages Processed (Rate)",
    "type": "graph",
    "targets": [
      {
        "expr": "rate(messages_processed_total[5m])",
        "legendFormat": "{{pipeline}} - {{status}}"
      }
    ]
  },
  {
    "id": 2,
    "title": "Processing Duration (p99)",
    "type": "graph",
    "targets": [
      {
        "expr": "histogram_quantile(0.99,
rate(processing_duration_seconds_bucket[5m]))",
        "legendFormat": "{{pipeline}} - p99"
      }
    ]
  },
  {
    "id": 3,
    "title": "Kafka Consumer Lag",
    "type": "graph",
    "targets": [
      {
        "expr": "kafka_consumer_group_lag",
        "legendFormat": "{{group}} - {{topic}}"
      }
    ]
  },
  {
    "id": 4,
    "title": "Flink Job Status",
    "type": "stat",
    "targets": [
      {
        "expr": "flink_jobmanager_job_uptime",
        "legendFormat": "{{job_name}}"
      }
    ]
  }
]
}

```

8. Error Handling & Recovery {#error-handling}

Retry Strategies

File: retry_handler.py

```

import time
import logging
from functools import wraps
from typing import Callable, Type, Tuple

logger = logging.getLogger(__name__)

class RetryStrategy:
    """
    Configurable retry strategy
    """

    @staticmethod
    def exponential_backoff(attempt: int, base_delay: float = 1.0, max_delay:
float = 60.0) -> float:
        """Calculate exponential backoff delay"""
        delay = min(base_delay * (2 ** attempt), max_delay)
        return delay

    @staticmethod
    def linear_backoff(attempt: int, delay: float = 1.0) -> float:
        """Calculate linear backoff delay"""
        return delay * attempt

    @staticmethod
    def fixed_delay(attempt: int, delay: float = 1.0) -> float:
        """Fixed delay between retries"""
        return delay

def retry_with_backoff(
    max_attempts: int = 3,
    exceptions: Tuple[Type[Exception], ...] = (Exception,),
    backoff_strategy: Callable[[int], float] = RetryStrategy.exponential_backoff,
    on_retry: Callable[[Exception, int], None] = None
):
    """
    Decorator for retrying functions with backoff

    Args:
        max_attempts: Maximum number of retry attempts
        exceptions: Tuple of exceptions to catch and retry
        backoff_strategy: Function to calculate retry delay
        on_retry: Callback function called on each retry
    """

    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            last_exception = None

            for attempt in range(max_attempts):
                try:

```

```

        return func(*args, **kwargs)

    except exceptions as e:
        last_exception = e

    if attempt < max_attempts - 1:
        delay = backoff_strategy(attempt)
        logger.warning(
            f"Attempt {attempt + 1}/{max_attempts} failed for
{func.__name__}: {e}. "
            f"Retrying in {delay:.2f}s..."
        )

        # Call retry callback
        if on_retry:
            on_retry(e, attempt)

        time.sleep(delay)
    else:
        logger.error(
            f"All {max_attempts} attempts failed for
{func.__name__}: {e}"
        )

    # All retries exhausted
    raise last_exception

    return wrapper
return decorator

# Example usage
@retry_with_backoff(
    max_attempts=5,
    exceptions=(ConnectionError, TimeoutError),
    backoff_strategy=lambda attempt: RetryStrategy.exponential_backoff(attempt,
base_delay=2.0)
)
def fetch_data_from_api(url: str):
    """Fetch data with automatic retries"""
    import requests
    response = requests.get(url, timeout=10)
    response.raise_for_status()
    return response.json()

```

Dead Letter Queue (DLQ) Implementation

File: `dlq_handler.py`

```

from confluent_kafka import Producer, Consumer
import json
from datetime import datetime

```

```

from typing import Dict, Any
import logging

logger = logging.getLogger(__name__)

class DeadLetterQueueHandler:
    """
    Handle failed messages by sending to DLQ
    """

    def __init__(self, bootstrap_servers: str, dlq_topic_suffix: str = ".dlq"):
        self.bootstrap_servers = bootstrap_servers
        self.dlq_topic_suffix = dlq_topic_suffix

        # Initialize producer for DLQ
        self.dlq_producer = Producer({
            'bootstrap.servers': bootstrap_servers,
            'client.id': 'dlq-producer'
        })

    def send_to_dlq(self, original_topic: str, message: Any, error: Exception,
        metadata: Dict = None):
        """
        Send failed message to Dead Letter Queue

        Args:
            original_topic: Original topic name
            message: Original message that failed
            error: Exception that caused the failure
            metadata: Additional metadata
        """
        dlq_topic = f"{original_topic}{self.dlq_topic_suffix}"

        dlq_message = {
            'original_topic': original_topic,
            'original_message': message,
            'error_type': type(error).__name__,
            'error_message': str(error),
            'timestamp': datetime.now().isoformat(),
            'metadata': metadata or {}
        }

        try:
            self.dlq_producer.produce(
                topic=dlq_topic,
                value=json.dumps(dlq_message).encode('utf-8'),
                callback=self._delivery_callback
            )
            self.dlq_producer.poll(0)

            logger.info(f"Message sent to DLQ: {dlq_topic}")

        except Exception as e:
            logger.error(f"Failed to send to DLQ: {e}")

```

```

        # Fallback: write to local disk
        self._write_to_disk(dlq_message)

def _delivery_callback(self, err, msg):
    """Delivery report callback for DLQ messages"""
    if err:
        logger.error(f"DLQ message delivery failed: {err}")
    else:
        logger.debug(f"DLQ message delivered to {msg.topic()}")

def _write_to_disk(self, message: Dict):
    """Fallback: write to local disk"""
    import os
    from pathlib import Path

    dlq_dir = Path("dlq_fallback")
    dlq_dir.mkdir(exist_ok=True)

    filename = f"dlq_{int(datetime.now().timestamp() * 1000)}.json"
    filepath = dlq_dir / filename

    with open(filepath, 'w') as f:
        json.dump(message, f, indent=2)

    logger.info(f"DLQ message written to disk: {filepath}")

def process_dlq(self, dlq_topic: str, processor: Callable[[Dict], None]):
    """
    Process messages from DLQ

    Args:
        dlq_topic: DLQ topic name
        processor: Function to process DLQ messages
    """
    consumer = Consumer({
        'bootstrap.servers': self.bootstrap_servers,
        'group.id': 'dlq-processor',
        'auto.offset.reset': 'earliest'
    })

    consumer.subscribe([dlq_topic])

    try:
        while True:
            msg = consumer.poll(1.0)

            if msg is None:
                continue
            if msg.error():
                logger.error(f"Consumer error: {msg.error()}")
                continue

            # Parse DLQ message
            dlq_message = json.loads(msg.value().decode('utf-8'))

```

```

        try:
            # Process DLQ message
            processor(dlq_message)
            consumer.commit(msg)
            logger.info("DLQ message processed successfully")

        except Exception as e:
            logger.error(f"Failed to process DLQ message: {e}")
            # Optionally: send to secondary DLQ or alert

    except KeyboardInterrupt:
        logger.info("Stopping DLQ processor...")
    finally:
        consumer.close()

    def close(self):
        """Close DLQ handler"""
        self.dlq_producer.flush()

# Example usage
if __name__ == "__main__":
    dlq_handler = DeadLetterQueueHandler('localhost:9092')

    # Send to DLQ
    try:
        # Some processing that might fail
        raise ValueError("Processing failed")
    except Exception as e:
        dlq_handler.send_to_dlq(
            original_topic='user-events',
            message={'user_id': '123', 'event': 'click'},
            error=e,
            metadata={'attempt': 3}
        )

    dlq_handler.close()

```

9. Testing Strategies {#testing}

Unit Tests

File: `tests/test_kafka_producer.py`

```

import pytest
from unittest.mock import Mock, patch
from kafka_producer import RobustKafkaProducer

class TestKafkaProducer:

```

```

@pytest.fixture
def producer(self):
    """Create producer instance for testing"""
    with patch('kafka_producer.Producer'):
        producer = RobustKafkaProducer('localhost:9092')
        yield producer
        producer.close()

def test_send_message_success(self, producer):
    """Test successful message send"""
    message = {'user_id': '123', 'event': 'click'}

    # Mock delivery callback
    with patch.object(producer.producer, 'produce') as mock_produce:
        producer.send('test-topic', message, key='123')

        assert mock_produce.called
        assert mock_produce.call_count == 1

def test_send_message_with_retry(self, producer):
    """Test message send with retries"""
    message = {'user_id': '123', 'event': 'click'}

    # Mock producer to fail first, then succeed
    with patch.object(producer.producer, 'produce', side_effect=[BufferError,
None]):
        producer.send('test-topic', message, key='123')

        # Should have retried
        assert producer.producer.produce.call_count == 2

def test_send_to_dlq_on_failure(self, producer):
    """Test DLQ handling on failure"""
    message = {'user_id': '123', 'event': 'click'}

    with patch.object(producer, '_handle_send_error') as mock_dlq:
        with patch.object(producer.producer, 'produce',
side_effect=Exception("Network error")):
            with pytest.raises(Exception):
                producer.send('test-topic', message, key='123')

            # Should have attempted DLQ
            assert mock_dlq.called

```

Integration Tests

File: `tests/integration/test_pipeline.py`

```

import pytest
from testcontainers.kafka import KafkaContainer
from testcontainers.postgres import PostgresContainer

```

```

import time

@pytest.fixture(scope="module")
def kafka_container():
    """Start Kafka container for testing"""
    with KafkaContainer() as kafka:
        yield kafka

@pytest.fixture(scope="module")
def postgres_container():
    """Start PostgreSQL container for testing"""
    with PostgresContainer("postgres:15") as postgres:
        yield postgres

def test_end_to_end_pipeline(kafka_container, postgres_container):
    """Test entire pipeline end-to-end"""

    # 1. Produce messages to Kafka
    from kafka import KafkaProducer
    import json

    producer = KafkaProducer(
        bootstrap_servers=kafka_container.get_bootstrap_server(),
        value_serializer=lambda v: json.dumps(v).encode('utf-8')
    )

    messages = [
        {'user_id': f'user-{i}', 'event': 'click'}
        for i in range(100)
    ]

    for msg in messages:
        producer.send('test-events', msg)

    producer.flush()

    # 2. Start consumer/processor
    # (Your pipeline code here)

    # 3. Wait for processing
    time.sleep(5)

    # 4. Verify results in database
    import psycopg2

    conn = psycopg2.connect(
        host=postgres_container.get_container_host_ip(),
        port=postgres_container.get_exposed_port(5432),
        database="test",
        user="test",
        password="test"
    )

    cursor = conn.cursor()

```

```

cursor.execute("SELECT COUNT(*) FROM processed_events")
count = cursor.fetchone()[0]

assert count == 100, f"Expected 100 events, found {count}"

```

Load Tests

File: tests/load/locustfile.py

```

from locust import User, task, between
from confluent_kafka import Producer
import json
import random

class KafkaUser(User):
    """
    Locust user for load testing Kafka producers
    """
    wait_time = between(0.1, 0.5)

    def on_start(self):
        """Initialize Kafka producer"""
        self.producer = Producer({
            'bootstrap.servers': 'localhost:9092',
            'client.id': f'locust-{self.__hash__()}'
        })

    @task(weight=10)
    def send_user_event(self):
        """Send user event"""
        event = {
            'user_id': f'user-{random.randint(1, 10000)}',
            'event_type': random.choice(['click', 'view', 'purchase']),
            'timestamp': int(time.time() * 1000)
        }

        self.producer.produce(
            'user-events',
            value=json.dumps(event).encode('utf-8'),
            callback=self.delivery_callback
        )
        self.producer.poll(0)

    @task(weight=5)
    def send_large_event(self):
        """Send large event (test throughput)"""
        event = {
            'user_id': f'user-{random.randint(1, 10000)}',
            'event_type': 'large_payload',
            'data': 'x' * 10000 # 10KB payload
        }

```

```

        self.producer.produce(
            'large-events',
            value=json.dumps(event).encode('utf-8')
        )
        self.producer.poll(0)

    def delivery_callback(self, err, msg):
        """Track delivery success/failure"""
        if err:
            self.environment.events.request_failure.fire(
                request_type="kafka",
                name="produce",
                response_time=0,
                exception=err
            )
        else:
            self.environment.events.request_success.fire(
                request_type="kafka",
                name="produce",
                response_time=0,
                response_length=len(msg.value())
            )

    def on_stop(self):
        """Cleanup"""
        self.producer.flush()

```

Run load test:

```

# Install locust
pip install locust

# Run load test
locust -f locustfile.py --host=localhost:9092

# Or headless mode
locust -f locustfile.py \
    --host=localhost:9092 \
    --users 100 \
    --spawn-rate 10 \
    --run-time 10m \
    --headless

```

10. CI/CD Pipelines {#cicd}

GitHub Actions Workflow

File: [.github/workflows/ci-cd.yml](#)

```
name: Data Pipeline CI/CD

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

env:
  DOCKER_REGISTRY: ghcr.io
  IMAGE_NAME: ${github.repository}/data-pipeline

jobs:

  # Job 1: Lint and test
  test:
    runs-on: ubuntu-latest

    services:
      kafka:
        image: confluentinc/cp-kafka:7.5.0
        env:
          KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
          KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
        ports:
          - 9092:9092

      postgres:
        image: postgres:15
        env:
          POSTGRES_PASSWORD: postgres
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Cache dependencies
        uses: actions/cache@v3
        with:
          path: ~/.cache/pip
```

```

    key: ${{ runner.os }}-pip-${{ hashFiles('**/requirements.txt') }}

- name: Install dependencies
  run: |
    python -m pip install --upgrade pip
    pip install -r requirements.txt
    pip install -r requirements-dev.txt

- name: Lint with flake8
  run: |
    flake8 . --count --select=E9,F63,F7,F82 --show-source --statistics
    flake8 . --count --max-complexity=10 --max-line-length=127 --statistics

- name: Type check with mypy
  run: mypy src/

- name: Run unit tests
  run: pytest tests/unit/ -v --cov=src --cov-report=xml

- name: Run integration tests
  run: pytest tests/integration/ -v
  env:
    KAFKA_BOOTSTRAP_SERVERS: localhost:9092
    POSTGRES_HOST: localhost

- name: Upload coverage
  uses: codecov/codecov-action@v3
  with:
    file: ./coverage.xml

# Job 2: Build and push Docker image
build:
  runs-on: ubuntu-latest
  needs: test
  if: github.event_name == 'push'

  steps:
    - name: Checkout code
      uses: actions/checkout@v3

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v2

    - name: Log in to Container Registry
      uses: docker/login-action@v2
      with:
        registry: ${{ env.DOCKER_REGISTRY }}
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}

    - name: Extract metadata
      id: meta
      uses: docker/metadata-action@v4
      with:

```

```

        images: ${ env.DOCKER_REGISTRY }}/${ env.IMAGE_NAME }}
        tags: |
            type=ref,event=branch
            type=sha,prefix={{branch}}-
            type=semver,pattern={{version}}

- name: Build and push
  uses: docker/build-push-action@v4
  with:
    context: .
    push: true
    tags: ${ steps.meta.outputs.tags }}
    labels: ${ steps.meta.outputs.labels }}
    cache-from: type=gha
    cache-to: type=gha,mode=max

# Job 3: Deploy to staging
deploy-staging:
  runs-on: ubuntu-latest
  needs: build
  if: github.ref == 'refs/heads/develop'
  environment: staging

  steps:
    - name: Deploy to Kubernetes (Staging)
      uses: azure/k8s-deploy@v4
      with:
        manifests: |
          k8s/staging/deployment.yaml
          k8s/staging/service.yaml
        images: |
          ${ env.DOCKER_REGISTRY }}/${ env.IMAGE_NAME }}:develop-${ env
github.sha }}
        kubectl-version: 'latest'

# Job 4: Deploy to production
deploy-production:
  runs-on: ubuntu-latest
  needs: build
  if: github.ref == 'refs/heads/main'
  environment: production

  steps:
    - name: Deploy to Kubernetes (Production)
      uses: azure/k8s-deploy@v4
      with:
        manifests: |
          k8s/production/deployment.yaml
          k8s/production/service.yaml
        images: |
          ${ env.DOCKER_REGISTRY }}/${ env.IMAGE_NAME }}:main-${ env github.sha
}}

        kubectl-version: 'latest'

```

```
- name: Notify deployment
  uses: slackapi/slack-github-action@v1.24.0
  with:
    payload: |
      {
        "text": "Data pipeline deployed to production",
        "blocks": [
          {
            "type": "section",
            "text": {
              "type": "mrkdown",
              "text": ":rocket: *Data Pipeline Deployed* \n\n*Environment:*
Production\n*Version:* ${ github.sha }\n*By:* ${ github.actor }"
            }
          }
        ]
      }
    env:
      SLACK_WEBHOOK_URL: ${ secrets.SLACK_WEBHOOK }
```

Conclusion

This implementation guide provides production-ready code for all major components of a data ingestion system. Key takeaways:

1. **Start with Docker Compose** for local development
2. **Implement comprehensive error handling** with retries and DLQs
3. **Add monitoring from day one** with Prometheus/Grafana
4. **Write tests** at all levels (unit, integration, load)
5. **Automate deployments** with CI/CD pipelines

Document Complete Version: 1.0